The three MABEL arm controllers, validated on hardware

Pure PD • PD + gravity compensation • Compliant drag-and-hold

`controller/mabel/arm_control` — on the real left arm

2026-07-14

What was tested

Three controllers, each a parameterisation of the DaMiao MIT law $T = k_p(q^* - q) + k_d(\dot{q}^* - \dot{q}) + t_{\text{ff}}$ that the arm firmware runs, all living in `controller/mabel/arm_control` and consumed identically by the sim, the hardware bridge, and this experiment:

controller	parameters	intent
PURE_PD	k_p full, $t_{\text{ff}} = 0$	Day-1 baseline; droops under gravity
GRAVITY_PD	k_p full, $t_{\text{ff}} = \tau_g(q)$	cancels gravity → tracks
COMPLIANT	$k_p \approx 0$, small k_d, $t_{\text{ff}} = \tau_g(q)$	drag it; gravity holds it

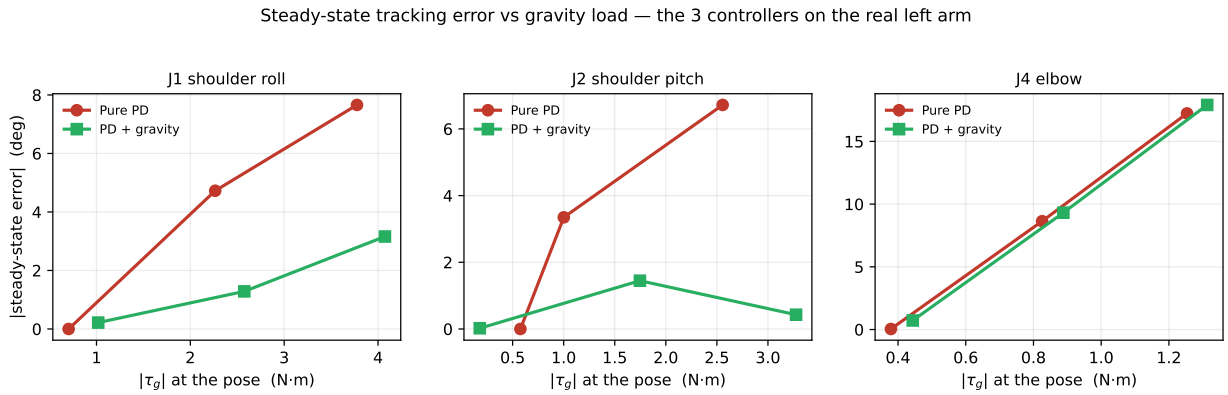
The gravity torque $\tau_g(q)$ is computed by `ArmGravityCompensator` on the canonical MJCF with the ORCA hand mass *stripped* (the hands are off the robot), in the raw motor frame. Tested on the most gravity-loaded joints — J1 (shoulder roll), J2 (shoulder pitch), J4 (elbow) — at small angles ($|\Delta| \leq 0.30$ rad from rest), each target held 12 s, steady state = median of the last 3 s. Targets were sent over the `joint_command` wire (the iOS path); the mode was switched live over `/arms/control_mode`.

Why pure PD must droop, and gravity comp must not

At a static hold $\dot{q} = 0$, so the motor supplies the joint's gravity torque: $k_p(q^* - q) + t_{\text{ff}} = \tau_g(q) \Rightarrow e_{\text{ss}} = (\tau_g - t_{\text{ff}})/k_p$. Pure PD ($t_{\text{ff}} = 0$) therefore droops $e = \tau_g/k_p$, growing with the load; and k_p cannot be raised out of it ($k_d \leq 5$ is a 12-bit hardware cap, which caps k_p). Gravity comp sets $t_{\text{ff}} = \tau_g$, so $e \to 0$ regardless of k_p.

Phase 1 & 2 — tracking (measured)

joint	target (deg)	$\lvert\tau_g\rvert$ (N·m)	Pure PD err	PD+gravity err
J1	-3	0.70	+0.0°	−0.2°
	-12	2.27	+4.7°	−1.3°
	-20	3.78	+7.7°	−3.2°
J2	-3	0.58	+0.0°	+0.0°
	+6	1.00	−3.3°	−1.4°
	+14	2.56	−6.7°	+0.4°
J4	+87	0.38	+0.0°	+0.7°
	+79	0.83	+8.6°	+9.3°
	+70	1.25	+17.2°	+17.9°



Pure PD droops with the gravity load exactly as $e = \tau_g / k_p$ predicts (J1 to $+7.7°$, J2 to $-6.7°$). Adding the gravity feed-forward collapses it (J2 to $+0.4°$; J1 to $-3.2°$, a slight over-compensation discussed below). J4 sits against its travel limit at rest and cannot reach a downward target in *either* mode, so its error is mechanical, not a controller property — it is shown for completeness.

Phase 3 — compliance (measured)

With COMPLIANT engaged ($k_p \approx 0$, gravity fed forward, small viscous k_d) and *no* position target, the arm held itself against gravity with a maximum drift of $0.5°$ (J1) and $< 0.05°$ (J2, J4) over 14 s. Two things follow: the arm does *not* fall (gravity comp carries its weight) and, because $k_p \approx 0$, it pulls toward no target — it is back-drivable. That is the "push it anywhere and it stays, as if gravity were off" behaviour; the operator's hand-drag is the final acceptance test.

Notes / open tuning

The J1 PD+gravity error is a slight *over*-compensation ($-3.2°$), the same signature seen on the shoulder-pitch study: the model lifts marginally too hard. The one open knob is the shoulder-mount tilt (metal $0°$ vs. MJCF $-10°$) and the hands-off mass model; both are explicit and trimmable (`MABEL_ARMS_MOUNT_TILT_DEG`, `HandsConfig`, `FF_SCALE`). Compliance damping k_d (`ComplianceTuning.kd_scale/k` is the knob to dial in on the hand-drag: raise it if a nudged joint coasts, lower it if the arm feels sticky.